

lab02

September 22, 2026

1 Tạo Tensor cơ bản

```
[2]: import torch

a = torch.tensor(
    [[1,2,3],
     [4,5,6]],
    dtype=torch.int32
)

print(a)
```

```
tensor([[1, 2, 3],
        [4, 5, 6]], dtype=torch.int32)
```

- kích thước: 2x3
- số chiều: 2
- kiểu dữ liệu: torch.int32 (số nguyên 32-bit)

2 Tạo các Tensor thường dùng

```
[3]: a = torch.rand(3, 4)
b = torch.randn(3, 4)
c = torch.zeros(2, 3)
d = torch.ones(2, 3)
e = torch.eye(4)

print(a)
print(b)
print(c)
print(d)
print(e)
```

```
tensor([[0.6394, 0.6253, 0.2941, 0.6399],
        [0.7686, 0.3819, 0.3924, 0.7104],
        [0.2206, 0.7069, 0.3688, 0.5964]])
tensor([[ 1.1918, -1.0460,  0.1348, -0.8319],
        [-0.2305, -1.6950, -1.4112,  0.0580],
```

```

        [ 0.4068,  1.7889, -0.0441,  0.1701]])
tensor([[0., 0., 0.],
        [0., 0., 0.]])
tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[1., 0., 0., 0.],
        [0., 1., 0., 0.],
        [0., 0., 1., 0.],
        [0., 0., 0., 1.]])

```

2.0.1 Sự khác nhau giữa torch.rand() và torch.randn():

Đặc điểm	torch.rand()	torch.randn()
Phân phối	Phân phối đều	Phân phối chuẩn hóa
Khoảng giá trị	[0,1)	$(-\infty, +\infty)$
Giá trị trung bình	≈ 0.5	≈ 0
Độ lệch chuẩn	≈ 0.289	1
Ký tự nhận biết	rand = random	randn = random normal

2.0.2 Tạo thêm một tensor 5x5 chứa toàn giá trị 10

```

[5]: x = torch.full((5,5), 10)
     print(x)

```

```

tensor([[10, 10, 10, 10, 10],
        [10, 10, 10, 10, 10],
        [10, 10, 10, 10, 10],
        [10, 10, 10, 10, 10],
        [10, 10, 10, 10, 10]])

```

3 Kiểm tra thông tin Tensor

```

[7]: a = torch.zeros(2,3,4)
     print("Tensor: ")
     print("Shape: ", a.shape)
     print("Size: ", a.size())
     print("Number of dimensions: ", a.ndim)
     print("Data type: ", a.dtype)
     print(a)

```

```

Tensor:
Shape: torch.Size([2, 3, 4])
Size: torch.Size([2, 3, 4])
Number of dimensions: 3
Data type: torch.float32
tensor([[[0., 0., 0., 0.],

```

```

[0., 0., 0., 0.],
[0., 0., 0., 0.]],

[[0., 0., 0., 0.],
 [0., 0., 0., 0.],
 [0., 0., 0., 0.]])

```

- Ý nghĩa của `torch.Size([2, 3, 4])`: độ dày = 2, chiều cao = 3, chiều rộng = 4

4 Slicing và Indexing

```

[1]: import torch
a = torch.randn(3, 4, 5)
print("Original tensor:")
print(a)
print("\nTensor a[1]:")
print(a[1])
print("\nTensor a[1:, 2:4]:")
print(a[1:, 2:4])

```

Original tensor:

```

tensor([[[ 1.9330e+00,  7.0211e-01,  3.1436e-01, -2.4266e-01, -6.2170e-01],
          [ 8.8653e-01,  4.9670e-01, -2.7772e-01, -8.7878e-01,  3.5992e-01],
          [-1.4606e+00, -1.3768e+00, -8.6951e-01,  6.1727e-01,  1.3598e-01],
          [ 1.1935e+00,  1.0362e-01,  1.2242e+00,  1.8752e-02, -6.8381e-01]],

        [[-2.4315e-02, -9.0540e-01,  1.1239e+00, -5.1844e-01,  1.0776e-01],
          [-7.0421e-01,  2.3597e-01, -8.7637e-02, -2.3892e-01, -2.4323e+00],
          [-2.0400e+00, -5.0846e-01,  5.3313e-01,  1.7817e+00,  1.5758e+00],
          [-8.3396e-01,  1.4261e+00, -1.8645e-01,  5.2642e-02,  7.2700e-01]],

        [[-1.0344e+00,  1.2407e-01, -5.0450e-02, -1.6198e+00,  1.7026e+00],
          [-3.7933e-02,  4.8985e-01,  1.5886e+00,  9.8663e-01, -2.7739e-01],
          [ 4.3803e-02, -1.1646e+00, -7.1749e-01, -4.8664e-01, -1.4200e+00],
          [ 2.2686e-03, -2.7415e-01,  9.7512e-01,  9.5183e-02, -5.2356e-01]])

```

Tensor a[1]:

```

tensor([[ -0.0243, -0.9054,  1.1239, -0.5184,  0.1078],
        [-0.7042,  0.2360, -0.0876, -0.2389, -2.4323],
        [-2.0400, -0.5085,  0.5331,  1.7817,  1.5758],
        [-0.8340,  1.4261, -0.1865,  0.0526,  0.7270]])

```

Tensor a[1:, 2:4]:

```

tensor([[[ -2.0400, -0.5085,  0.5331,  1.7817,  1.5758],
          [-0.8340,  1.4261, -0.1865,  0.0526,  0.7270]],

        [[ 0.0438, -1.1646, -0.7175, -0.4866, -1.4200],
          [ 0.0023, -0.2741,  0.9751,  0.0952, -0.5236]])

```

```
[3]: print(a[1].shape)
```

```
torch.Size([4, 5])
```

```
[4]: print(a[1:,2:4].shape)
```

```
torch.Size([2, 2, 5])
```

```
[5]: result = a[:, :, -2:]  
print(result)
```

```
tensor([[[[-0.2427, -0.6217],  
          [-0.8788,  0.3599],  
          [ 0.6173,  0.1360],  
          [ 0.0188, -0.6838]],  
  
        [[-0.5184,  0.1078],  
          [-0.2389, -2.4323],  
          [ 1.7817,  1.5758],  
          [ 0.0526,  0.7270]],  
  
        [[-1.6198,  1.7026],  
          [ 0.9866, -0.2774],  
          [-0.4866, -1.4200],  
          [ 0.0952, -0.5236]]]])
```

5 Reshape và Flatten

```
[4]: import torch  
a = torch.randn(3, 4)  
print("Original:")  
print(a)  
print("\nFlatten:")  
print(a.ravel())  
print("\nReshape to 3 x 2 x 2:")  
print(a.reshape(3, 2, 2))
```

Original:

```
tensor([[ 0.1780, -1.0269,  0.2351,  1.6074],  
        [-1.2076,  0.8034, -0.3705,  0.3919],  
        [-1.9080, -0.0509, -2.0806, -0.0385]])
```

Flatten:

```
tensor([ 0.1780, -1.0269,  0.2351,  1.6074, -1.2076,  0.8034, -0.3705,  0.3919,  
        -1.9080, -0.0509, -2.0806, -0.0385])
```

Reshape to 3 x 2 x 2:

```
tensor([[[ 0.1780, -1.0269],
```

```

[ 0.2351,  1.6074]],

[[-1.2076,  0.8034],
 [-0.3705,  0.3919]],

[[-1.9080, -0.0509],
 [-2.0806, -0.0385]]])

```

5.0.1 Kiểm tra số phần tử trước và sau khi reshape

- Trước: $3 \times 4 = 12$ phần tử
- Sau: $2 \times 2 \times 3 = 12$ phần tử

5.0.2 Reshape tensor thành kích thước 2x6

```

[5]: a= torch.randn(3,4)
      print(a.reshape(2,6))

```

```

tensor([[[-1.5464, -0.1452,  0.9226,  0.0977,  0.1219, -0.3343],
          [ 0.0124, -0.6020, -1.2894, -0.3245, -0.6683, -0.3814]])

```

6 Phép toán nhân ma trận

```

[6]: import torch
      a = torch.randn(2, 3)
      b = torch.randn(2, 3)
      print("a:")
      print(a)
      print("b:")
      print(b)
      print("\na + b:")
      print(a + b)
      print("\na / b:")
      print(a / b)
      print("\na^2:")
      print(a ** 2)
      print("\nMatrix multiplication:")
      print(a @ b.T)

```

```

a:
tensor([[ 2.3518,  0.6959,  0.3171],
         [ 1.3669, -0.5038, -0.3444]])

```

```

b:
tensor([[ 0.1931,  1.4687, -0.1540],
         [-0.5237,  2.5670, -0.6786]])

```

```

a + b:
tensor([[ 2.5449,  2.1646,  0.1631],

```

```

[ 0.8432,  2.0632, -1.0230]])

a / b:
tensor([[12.1781,  0.4738, -2.0594],
        [-2.6099, -0.1963,  0.5075]])

a^2:
tensor([[5.5311, 0.4843, 0.1006],
        [1.8685, 0.2538, 0.1186]])

Matrix multiplication:
tensor([[ 1.4275,  0.3395],
        [-0.4229, -1.7754]])

```

6.0.1 Phân biệt phép toán $a * b$ với $a @ b.T$

- $a * b$: là nhân từng phần tử tương ứng tại cùng vị trí, a và b phải cùng kích thước
- $a @ b.T$: là phép nhân ma trận đại số tuyến tính - lấy hàng của a nhân vô hướng với cột của b^T , số cột của a phải bằng số hàng của $b.T$

6.0.2 Shape của kết quả $a @ b.T$

```

[7]: a_multiplication = a @ b.T
      print(a_multiplication.shape)

```

```
torch.Size([2, 2])
```

- Shape = [2,2]

7 Các hàm thống kê trên Tensor

```

[8]: import torch
      a = torch.randn(3, 4)
      print(a)
      print("\nMean:")
      print(torch.mean(a, dim=0))
      print("\nStandard deviation:")
      print(torch.std(a, dim=0))
      print("\nCumulative sum:")
      print(torch.cumsum(a, dim=0))

```

```

tensor([[ 1.0302, -1.4289, -0.6581,  0.8301],
        [-0.4182,  2.0628, -0.9782,  0.4673],
        [-1.0591, -0.7634,  0.9340, -1.3999]])

```

Mean:

```
tensor([-0.1490, -0.0432, -0.2341, -0.0342])
```

Standard deviation:

```
tensor([1.0703, 1.8539, 1.0242, 1.1966])
```

Cumulative sum:

```
tensor([[ 1.0302, -1.4289, -0.6581,  0.8301],
        [ 0.6120,  0.6339, -1.6363,  1.2974],
        [-0.4471, -0.1295, -0.7023, -0.1025]])
```

```
[9]: import torch
a = torch.randn(3, 4)
print(a)
print("\nMean:")
print(torch.mean(a, dim=1))
print("\nStandard deviation:")
print(torch.std(a, dim=1))
print("\nCumulative sum:")
print(torch.cumsum(a, dim=1))
```

```
tensor([[ 2.2106, -0.0630, -1.5784, -0.3669],
        [-0.3499,  0.2773, -0.7984,  0.4686],
        [-0.1831,  1.5771,  0.8041, -0.6978]])
```

Mean:

```
tensor([ 0.0506, -0.1006,  0.3751])
```

Standard deviation:

```
tensor([1.5818, 0.5819, 1.0151])
```

Cumulative sum:

```
tensor([[ 2.2106,  2.1476,  0.5693,  0.2024],
        [-0.3499, -0.0726, -0.8710, -0.4024],
        [-0.1831,  1.3940,  2.1981,  1.5003]])
```

7.0.1 Nhận xét sự thay đổi khi thay dim=0 và dim=1

- dim=0: Quét dọc từ trên xuống dưới qua 3 hàng cho mỗi cột, tính các phép tính theo từng cột
- dim=1: Quét ngang từ trái sang phải qua 4 cột cho mỗi hàng, tính các phép tính theo từng hàng

8 Autograd cơ bản

```
[10]: import torch
x = torch.tensor(
    3.6,
    requires_grad=True
)
y = x * x
```

```

y.backward()
print("x =", x)
print("y =", y)
print("x.grad =", x.grad)

```

```

x = tensor(3.6000, requires_grad=True)
y = tensor(12.9600, grad_fn=<MulBackward0>)
x.grad = tensor(7.2000)

```

8.0.1 Tính thủ công đạo hàm của $y = x^2$ tại $x = 3.6$ và so sánh với $x.grad$

- $y = f(x) = x^2$
- Đạo hàm bậc nhất của hàm số: $y' = \frac{dx}{dy} = 2x$
- Thay $x = 3.6$: $y'(3.6) = 2 \times 3.6 = 7.2$
- Vậy hai kết quả giống nhau

8.0.2 Đổi hàm thành $y = 3x^2 + 2x + 1$ và kiểm tra gradient

```

[12]: import torch
x = torch.tensor(
3.6,
requires_grad=True
)
y = 3*x**2 + 2*x + 1
y.backward()
print("x =", x)
print("y =", y)
print("x.grad =", x.grad)

```

```

x = tensor(3.6000, requires_grad=True)
y = tensor(47.0800, grad_fn=<AddBackward0>)
x.grad = tensor(23.6000)

```

9 Gradient Descent tìm hệ số đa thức

```

[14]: import numpy as np
import torch
polynomial = np.poly1d([1, 2, 3])
N = 20
X = np.random.randn(N, 1) * 5
Y = polynomial(X)
XX = np.hstack([
    X * X,
    X,
    np.ones_like(X)
])
w = torch.randn(3, 1, requires_grad=True)
x = torch.tensor(XX, dtype=torch.float32)

```

```

y = torch.tensor(Y, dtype=torch.float32)
optimizer = torch.optim.NAdam([w], lr=0.01)
print("Initial coefficients:")
print(w)
for epoch in range(1000):
    optimizer.zero_grad()
    y_pred = x @ w
    mse = torch.mean(torch.square(y - y_pred))
    mse.backward()
    optimizer.step()
print("Learned coefficients:")
print(w)

```

```

Initial coefficients:
tensor([[ -1.4686],
        [  0.0208],
        [-0.1785]], requires_grad=True)
Learned coefficients:
tensor([[ 1.0060],
        [ 2.0684],
        [ 2.4478]], requires_grad=True)

```

9.0.1 So sánh ba hệ số học được với [1, 2, 3]:

- Hệ số bậc hai: Hội tụ rất nhanh và chuẩn xác về 1
- Hệ số bậc một: Hội tụ tốt về giá trị 2
- Hệ số tự do: Thường hội tụ chậm hơn hai hệ số trên sau 1000 epoch

9.0.2 Giải thích vai trò của `optimizer.zero_grad()`, `mse.backward()` và `optimizer.step()`

- `optimizer.zero_grad()`: xóa gradient cũ về 0
- `mse.backward()`: lan truyền ngược - tính đạo hàm riêng của hàm mất mát đối với từng tham số có `requires_grad=True`
- `optimizer.step()`: cập nhật trọng số - điều chỉnh `w` theo hướng làm giảm hàm mất mát MSE nhất có thể cho vòng lặp tiếp theo

10 Giải hệ phương trình bằng Autograd

```

[17]: import random
import torch
A = torch.tensor(random.random(), requires_grad=True)
B = torch.tensor(random.random(), requires_grad=True)
C = torch.tensor(random.random(), requires_grad=True)
D = torch.tensor(random.random(), requires_grad=True)
EPOCHS = 2000
optimizer = torch.optim.NAdam([A, B, C, D], lr=0.01)

```

```

for _ in range(EPOCHS):
    y1 = A + B - 9
    y2 = C - D - 1
    y3 = A + C - 8
    y4 = B - D - 2
    sqerr = y1*y1 + y2*y2 + y3*y3 + y4*y4
    optimizer.zero_grad()
    sqerr.backward()
    optimizer.step()
print("A =", A)
print("B =", B)
print("C =", C)
print("D =", D)

```

```

A = tensor(4.4875, requires_grad=True)
B = tensor(4.5125, requires_grad=True)
C = tensor(3.5125, requires_grad=True)
D = tensor(2.5125, requires_grad=True)

```

10.0.1 Kiểm tra nghiệm thu được với bốn phương trình: $A+B=9$, $C-D=1$, $A+C=8$, $B-D=2$.

- Thỏa mãn gần như tuyệt đối (sai số ở bậc 10^{-} đến 10^{-5})

10.0.2 Nhận xét vì sao bài toán có thể có nhiều nghiệm.

- Hàm mục tiêu $sqerr(A, B, C, D)$ không có một điểm cực tiểu toàn cục duy nhất, mà tạo thành một đường thung lũng phẳng (valley) gồm các điểm mà tại đó $sqerr = 0$. Thuật toán NAdam xuất phát từ điểm ngẫu nhiên nào sẽ bị kéo về điểm nghiệm $sqerr = 0$ gần điểm xuất phát đó nhất theo hướng dốc của gradient.

11 Bài 11. Viết chương trình PyTorch để thực hiện các yêu cầu sau:

- Tạo Tensor X có 24 phần tử và reshape lần lượt thành 4×6 , 2×12 và $2 \times 3 \times 4$.
- Tạo hai Tensor A, B kích thước 3×4 ; tính $A+B$, $A*B$, A/B , trung bình theo cột và độ lệch chuẩn theo cột.
- Cho $y = 2x^2 + 5x + 3$. Dùng autograd tính dy/dx tại $x = 2$ và kiểm tra bằng phép tính thủ công.
- Thay đa thức trong Bài 9 bằng $y = 2x^2 - 3x + 5$; huấn luyện và cho biết các hệ số tìm được.

```

[18]: import numpy as np
import torch

torch.manual_seed(42)
np.random.seed(42)

print("==" * 60)

```

```

print("YÊU CẦU 1: TẠO VÀ RESHAPE TENSOR X")
print("=" * 60)

X = torch.arange(24)  # Tạo tensor gồm 24 số nguyên từ 0 đến 23
print(f"Tensor X ban đầu (shape {X.shape}): \n{X}\n")

X_4x6 = X.reshape(4, 6)
print(f"Reshape (4 x 6) - shape {X_4x6.shape}: \n{X_4x6}\n")

X_2x12 = X.reshape(2, 12)
print(f"Reshape (2 x 12) - shape {X_2x12.shape}: \n{X_2x12}\n")

X_2x3x4 = X.reshape(2, 3, 4)
print(f"Reshape (2 x 3 x 4) - shape {X_2x3x4.shape}: \n{X_2x3x4}\n")

print("=" * 60)
print("YÊU CẦU 2: CÁC PHÉP TOÁN TRÊN TENSOR A, B (3x4)")
print("=" * 60)

A = torch.randn(3, 4)
B = torch.randn(3, 4)

print("Tensor A:\n", A)
print("\nTensor B:\n", B)

print("\n1. Tổng (A + B):\n", A + B)
print("\n2. Tích từng phần tử (A * B):\n", A * B)
print("\n3. Thương từng phần tử (A / B):\n", A / B)

print("\n4. Trung bình theo cột của A (dim=0):\n", torch.mean(A, dim=0))
print("\n5. Độ lệch chuẩn theo cột của A (dim=0):\n", torch.std(A, dim=0))

print("YÊU CẦU 3: TÍNH ĐẠO HÀM BẰNG AUTOGRAD")
print("=" * 60)

x_val = torch.tensor(2.0, requires_grad=True)
y_val = 2 * x_val**2 + 5 * x_val + 3
y_val.backward()

print(f"Giá trị x = {x_val.item()}")
print(f"Giá trị y = {y_val.item()}")
print(f"Đạo hàm dy/dx bằng PyTorch Autograd: {x_val.grad.item()}")

print("\n" + "=" * 60)
print("YÊU CẦU 4: HỌC HỆ SỐ CỦA ĐA THỨC  $y = 2x^2 - 3x + 5$ ")
print("=" * 60)

```

```

polynomial = np.poly1d([2, -3, 5])
N = 20
X_poly = np.random.randn(N, 1) * 5
Y_poly = polynomial(X_poly)

XX = np.hstack([X_poly * X_poly, X_poly, np.ones_like(X_poly)])

x_train = torch.tensor(XX, dtype=torch.float32)
y_train = torch.tensor(Y_poly, dtype=torch.float32)

w = torch.randn(3, 1, requires_grad=True)
print(f"Hệ số khởi tạo ban đầu:\n{w.detach().numpy().flatten()}\n")

optimizer = torch.optim.NAdam([w], lr=0.05)

for epoch in range(3000):
    optimizer.zero_grad()
    y_pred = x_train @ w
    mse = torch.mean(torch.square(y_train - y_pred))
    mse.backward()
    optimizer.step()

learned_coeffs = w.detach().numpy().flatten()
print(f"Loss MSE sau huấn luyện: {mse.item():.6e}")
print(f"Các hệ số tìm được: {learned_coeffs}")
print(
    f"So sánh với hệ số gốc [2, -3, 5]: sai số tối đa <"
    f"{np.max(np.abs(learned_coeffs - np.array([2, -3, 5]))):.6f}"
)

```

=====

YÊU CẦU 1: TẠO VÀ RESHAPE TENSOR X

=====

Tensor X ban đầu (shape torch.Size([24])):

```

tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
        18, 19, 20, 21, 22, 23])

```

Reshape (4 x 6) - shape torch.Size([4, 6]):

```

tensor([[ 0,  1,  2,  3,  4,  5],
        [ 6,  7,  8,  9, 10, 11],
        [12, 13, 14, 15, 16, 17],
        [18, 19, 20, 21, 22, 23]])

```

Reshape (2 x 12) - shape torch.Size([2, 12]):

```

tensor([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11],
        [12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23]])

```

Reshape (2 x 3 x 4) - shape torch.Size([2, 3, 4]):

```

tensor([[[ 0,  1,  2,  3],
          [ 4,  5,  6,  7],
          [ 8,  9, 10, 11]],

        [[12, 13, 14, 15],
          [16, 17, 18, 19],
          [20, 21, 22, 23]]])

```

YÊU CẦU 2: CÁC PHÉP TOÁN TRÊN TENSOR A, B (3x4)

Tensor A:

```

tensor([[ 0.3367,  0.1288,  0.2345,  0.2303],
        [-1.1229, -0.1863,  2.2082, -0.6380],
        [ 0.4617,  0.2674,  0.5349,  0.8094]])

```

Tensor B:

```

tensor([[ 1.1103, -1.6898, -0.9890,  0.9580],
        [ 1.3221,  0.8172, -0.7658, -0.7506],
        [ 1.3525,  0.6863, -0.3278,  0.7950]])

```

1. Tổng (A + B):

```

tensor([[ 1.4470, -1.5610, -0.7545,  1.1883],
        [ 0.1993,  0.6309,  1.4424, -1.3886],
        [ 1.8142,  0.9537,  0.2071,  1.6043]])

```

2. Tích từng phần tử (A * B):

```

tensor([[ 0.3738, -0.2177, -0.2319,  0.2207],
        [-1.4846, -0.1523, -1.6911,  0.4789],
        [ 0.6244,  0.1835, -0.1753,  0.6434]])

```

3. Thương từng phần tử (A / B):

```

tensor([[ 0.3032, -0.0762, -0.2371,  0.2404],
        [-0.8493, -0.2280, -2.8834,  0.8500],
        [ 0.3413,  0.3895, -1.6320,  1.0181]])

```

4. Trung bình theo cột của A (dim=0):

```

tensor([-0.1082,  0.0699,  0.9925,  0.1339])

```

5. Độ lệch chuẩn theo cột của A (dim=0):

```

tensor([0.8810, 0.2325, 1.0635, 0.7285])

```

YÊU CẦU 3: TÍNH ĐẠO HÀM BẰNG AUTOGRAĐ

Giá trị x = 2.0

Giá trị y = 21.0

Đạo hàm dy/dx bằng PyTorch Autograd: 13.0

=====

YÊU CẦU 4: HỌC HỆ SỐ CỦA ĐA THỨC $y = 2x^2 - 3x + 5$

=====

Hệ số khởi tạo ban đầu:

[0.28151956 0.05616354 0.52271605]

Loss MSE sau huấn luyện: 3.946525e-10

Các hệ số tìm được: [2.0000007 -2.9999988 4.9999757]

So sánh với hệ số gốc [2, -3, 5]: sai số tối đa < 0.000024